YENEPOYA

(DEEMED TO BE UNIVERSITY)

Project – High Level Design

on

AI-Based Edge Healthcare Monitoring System

Student Name: Muhammed Fazin T.K

Industry Mentor: **Guided by:**

Table of Contents

1. Introduction. 6
   1.1. Scope of the document. 6
   1.2. Intended Audience.... 6
   1.3. System overview.... 6

2. System Design. 6
   2.1. Application Design.... 6
   2.2. Process Flow.... 6
   2.3. Information Flow. 7
   2.4. Components Design 8
   2.5. Key Design Considerations 8
   2.6. API Catalogue. 8

3. Data Design.... 8
   3.1. Data Model.... 8
   3.2. Data Access Mechanism.... 9
   3.3. Data Retention Policies 9
   3.4. Data Migration 9

4. Interfaces.... 9

5. State and Session Management 10

6. Caching 10

7. Non-Functional Requirements 10
   7.1. Security Aspects 10
   7.2. Performance Aspects 10

8. References 11

1. Introduction

The AI-Based Edge Healthcare Monitoring System is designed to perform real-time object detection using a webcam and a lightweight deep learning model. The system leverages Edge AI principles, where all computations are performed locally on the device instead of relying on cloud infrastructure. This ensures reduced latency, improved privacy, and uninterrupted operation even in offline environments.

In modern healthcare environments, continuous monitoring is essential for ensuring patient safety and operational efficiency. Traditional monitoring systems often depend on centralized servers, which introduce delays and security risks. This project addresses these limitations by implementing a decentralized, real-time monitoring system.

The system integrates computer vision techniques with a YOLO-based object detection model to identify objects such as persons, equipment, and other relevant entities. The results are displayed through a dashboard interface, providing real-time insights and logging capabilities.

1.1 Scope of the Document

This document provides a comprehensive description of the system at both conceptual and implementation levels. It includes system architecture, module design, data handling mechanisms, and performance considerations.

The scope covers:

- System architecture and workflow
- Module-level design and responsibilities
- Data structures and storage mechanisms
- User interface and interaction
- Performance and security considerations

This document serves as a technical reference for developers, evaluators, and stakeholders.

1.2 Intended Audience

The intended audience for this document includes:

- **Developers:** To understand system implementation and extend functionality
- **Project Evaluators:** To assess system design and performance
- **Technical Reviewers:** To validate architecture and design decisions
- **Students and Researchers:** To learn about Edge AI applications

1.3 System Overview

The system operates as a real-time object detection pipeline. It captures video input through a webcam, processes frames using a YOLO model, and generates predictions in the form of bounding boxes, labels, and confidence scores.

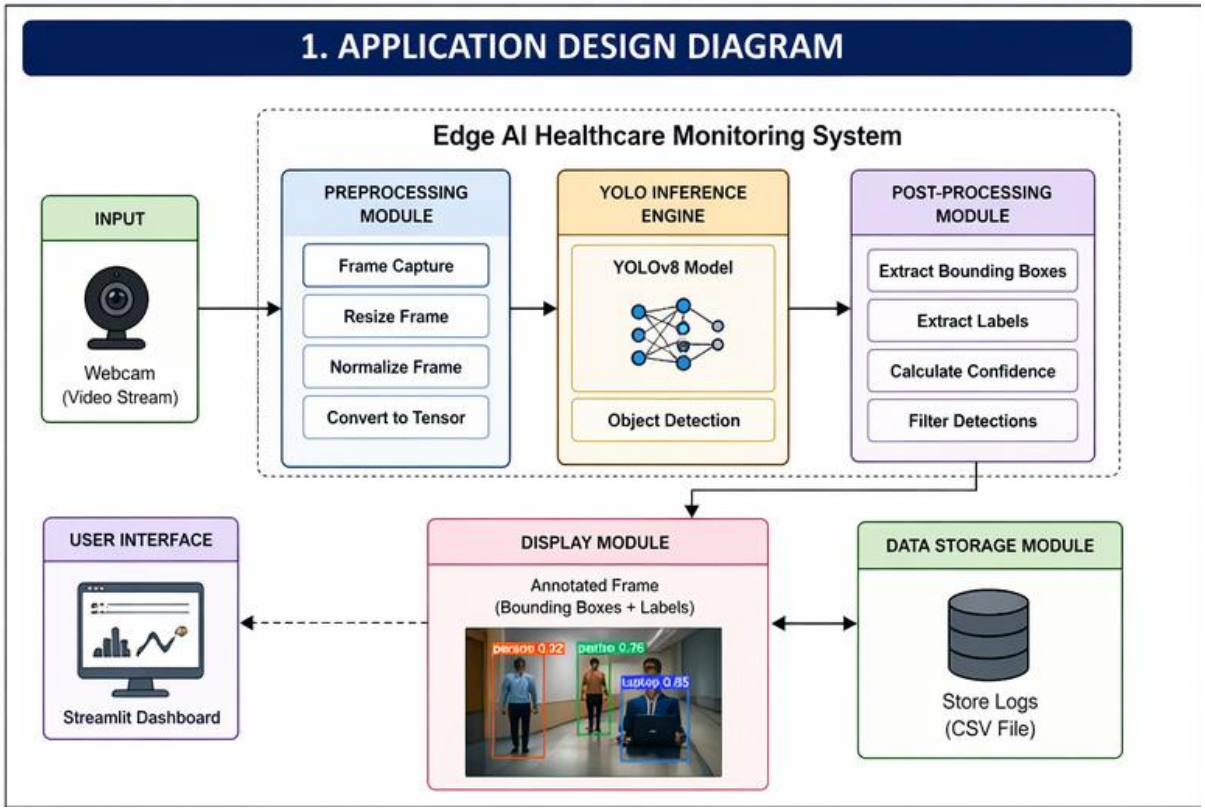
The processed results are displayed on a dashboard and stored for analysis. The system is capable of detecting multiple objects simultaneously and updating results dynamically.

Key features include:

- Real-time object detection
- Edge-based processing (no cloud dependency)
- Interactive dashboard interface
- Data logging and visualization

2. System Design

2.1 Application Design



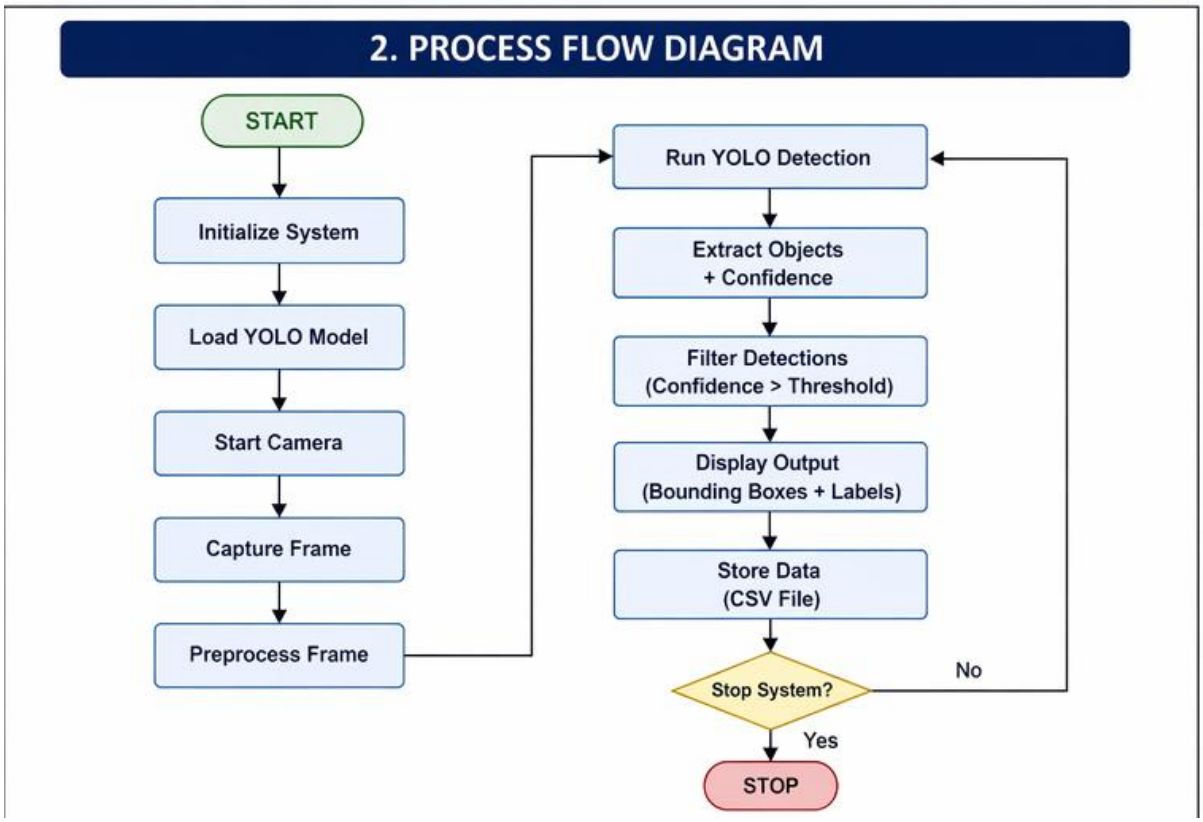
The application is designed using a modular architecture to ensure flexibility and maintainability. Each module performs a specific function and communicates with other modules through well-defined interfaces.

The system is divided into the following layers:

- **Input Layer:** Captures real-time video frames from the webcam
- **Preprocessing Layer:** Converts frames into a suitable format for the model
- **Inference Layer:** Executes the YOLO model to detect objects
- **Output Layer:** Displays results and stores data

This layered approach simplifies debugging, testing, and future enhancements.

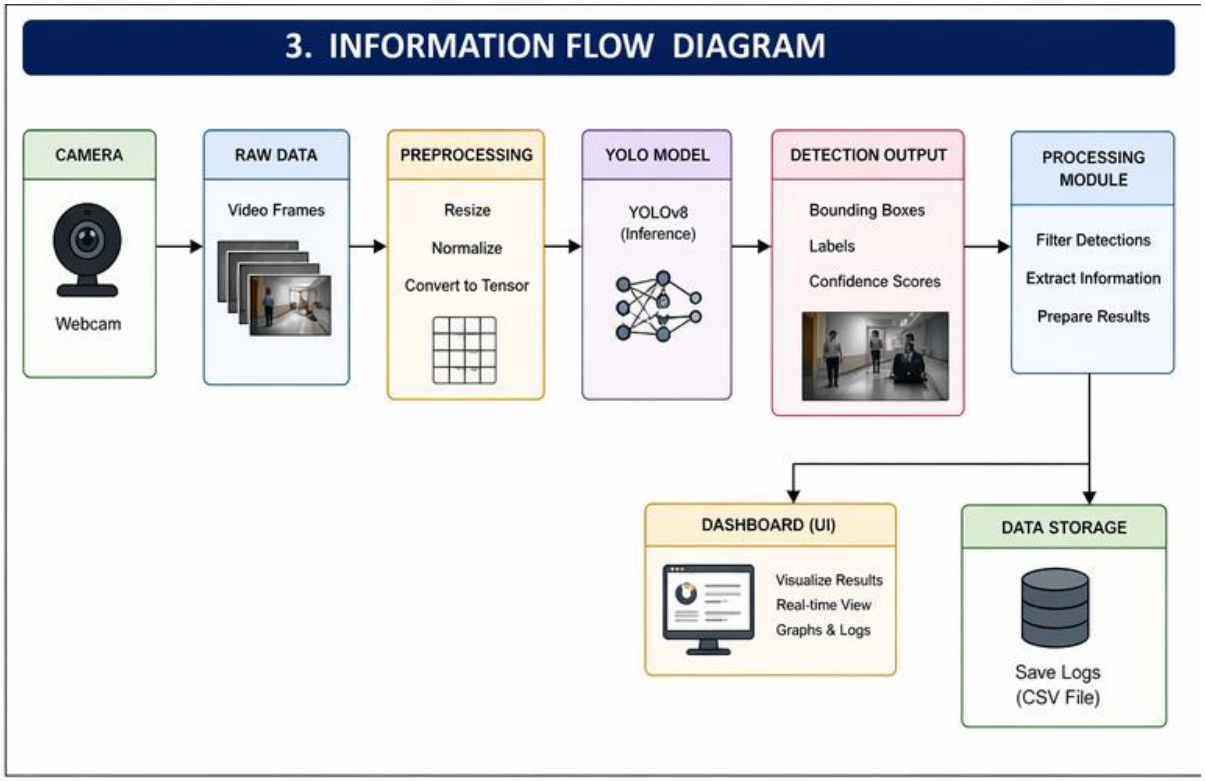
2.3 Process Flow



The system follows a continuous loop for processing frames. The detailed process flow is as follows:

1. Initialize the system and load the YOLO model
2. Start video capture from the webcam
3. Capture a frame from the video stream
4. Resize and preprocess the frame
5. Pass the frame to the YOLO model
6. Perform object detection
7. Extract bounding boxes, labels, and confidence scores
8. Filter detections based on threshold values
9. Display annotated frame on the dashboard
10. Log detection data into a CSV file
11. Repeat the process until the system is stopped

2.3 Information Flow

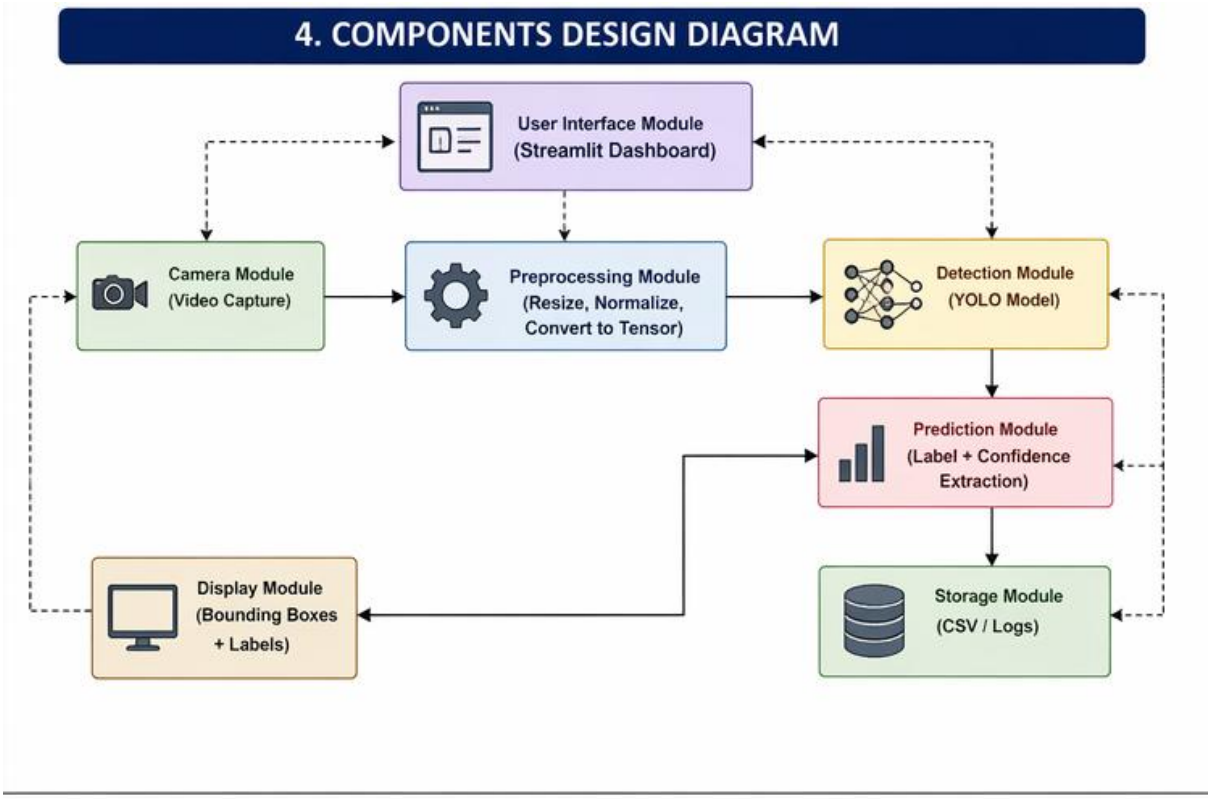


The information flow in the system defines how data moves between components:

- The webcam captures raw image data
- The preprocessing module transforms the data into a tensor format
- The inference module processes the tensor and produces predictions
- The prediction module extracts meaningful information
- The output is sent to the display and storage modules

This structured flow ensures efficient data handling and minimal processing delays.

2.4 Components Design



The system is composed of multiple components, each responsible for a specific function:

Camera Module

Captures real-time video frames using OpenCV. It ensures consistent frame acquisition and handles camera initialization and release.

Preprocessing Module

Processes raw frames by resizing, normalizing, and converting them into a format suitable for the model.

Detection Module

Uses the YOLO model to perform object detection. It identifies objects and generates bounding boxes and confidence scores.

Prediction Module

Interprets model output and extracts relevant information such as object labels and probabilities.

Display Module

Renders the processed frames and detection results on the dashboard interface.

Storage Module

Stores detection data in a structured format (CSV) for analysis and reporting.

2.5 Key Design Considerations

The system design is influenced by several key factors:

- **Performance:** Ensuring real-time processing capability
- **Accuracy:** Using a reliable detection model
- **Scalability:** Allowing future integration with additional features
- **Usability:** Providing an intuitive interface
- **Reliability:** Ensuring stable operation under different conditions

2.6 API Catalogue

API Name	Description
capture_frame()	Captures video frame
preprocess_frame()	Prepares frame for model
run_detection()	Executes YOLO model
extract_results()	Extracts labels and confidence
display_output()	Shows results
save_data()	Stores detection logs

3. Data Design

3.1 Data Model

Field Name	Description
Timestamp	Time of detection
Object	Detected object label
Confidence	Detection accuracy

3.2 Data Access Mechanism

Data is stored in CSV format and accessed using Python file handling libraries. The system appends new records dynamically during execution.

3.3 Data Retention Policies

- Data is stored locally
- No sensitive personal data is collected
- Users can delete or archive data manually

3.4 Data Migration

Data can be exported to other systems such as databases or analytics tools. Future enhancements may include automated data migration.

4. Interfaces

The system provides a graphical interface using Streamlit. It includes:

- Live video feed
- Detection results
- Data table
- Charts and analytics

5. State and Session Management

The system uses session state management to control execution flow. It allows users to start and stop the camera without restarting the application.

6. Caching

Currently, caching is not implemented. However, caching mechanisms can be introduced to improve performance for repeated operations.

7. Non-Functional Requirements

7.1 Security Aspects

- Data is processed locally
- No external data transmission
- Ensures privacy and confidentiality

7.2 Performance Aspects

- Real-time detection capability
- Low latency processing
- Efficient CPU utilization

8. References

- YOLO Documentation
- OpenCV Documentation
- Streamlit Documentation
- Python Official Documentation